

UNIVERSIDAD DEL VALLE DE GUATEMALA
FACULTAD DE INGENIERÍA



Usabilidad en Lenguajes de Programación de Dominio Específico (DSL): El caso de Nix

(Version Alpha)

Trabajo de graduación presentado por Daniel Alfredo Rayo Roldán para optar al grado académico en
Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Índice

1. Resumen	1
2. Introducción	1
3. Objetivos	2
3.a. General	2
3.b. Específicos	2
4. Marco Teórico	3
Referencias	3

1. Resumen

El problema de llevar software de una computadora a otra y que siga funcionando (proceso conocido como *Despliegue de Software* [1]) es un yugo con el que las Ciencias de Computación no han dado una solución definitiva, esencialmente porque para que una pieza de software funcione correctamente no solamente depende del código fuente en el que esta escrita, sino del contexto que le rodea (hardware, sistema operativo, dependencias, etc.), bajo ese contexto surgen Nix como un manejador de paquetes y sistema de construcción¹ que utiliza un enfoque inspirado en la pureza funcional, donde cada paquete define explícitamente el contexto en el que espera ser construido y ejecutado. Este enfoque a demostrado poder crear más de 700 mil artefactos binariamente identicos en diferentes computadoras [3] y contar con el repositorio de paquetes más grande a fecha de este documento [4]. La idea evolucionó al punto de definir el estado casi completo de un sistema operativo mediante un solo archivo de configuración [5].

Sin embargo, a pesar de sus capacidades prometedoras, Nix no ha gozado de la misma adopción que otras herramientas que abordan los mismos problemas de reproducibilidad como Docker, Conda o VirtualEnv u otros manejadores de paquetes [6]. Las causas de acuerdo a la comunidad son varias: Documentación compleja, errores cripticos, o un lenguaje de programación difícil de dominar; muchos de ellos no siendo problemas técnicos sino de experiencia de uso, y que cae en el rango de estudio del DX (*Developer Experience*) [7]. Este trabajo se enfocó en estudiar experiencia de desarrolladores utilizando el lenguaje de Nix, entendiendo sus problemas de usabilidad, y con lo aprendido desarrollar una librería para interactuar con Nix usando un lenguaje de proposito general.

2. Introducción

La computación moderna puede entenderse como un sistema altamente interdependiente, donde múltiples componentes —hardware, sistema operativo, bibliotecas, dependencias y configuraciones— deben operar de manera coordinada para garantizar el correcto funcionamiento del software. En este contexto, el desarrollo de aplicaciones suele realizarse dentro de un entorno específico y controlado, correspondiente a la máquina del desarrollador, que encapsula una combinación específica de las variables mencionadas [1].

Esta aparente estabilidad, sin embargo, resulta frágil al momento de transferir el software a un entorno distinto. Un fenómeno comúnmente observado es que una aplicación que funciona correctamente en el entorno original falla al ser ejecutada en otra máquina, incluso cuando el código fuente permanece inalterado, conocido informalmente como «*funciona en mi computadora*», evidencia que la ejecución de software no depende únicamente de su implementación, sino también del contexto en el que se despliega.

En consecuencia, el despliegue de software es un esfuerzo compartido entre de la aplicación y el sistema que la ejecuta, donde ambas partes hacen ciertas asumpciones sobre la otra. Ante este problema la solución predominante es empaquetar tanto la aplicación como el contexto en el que se espera que esta se ejecute, dando lugar a tecnologías de máquinas virtuales y contenedores [8], [9], [10]. Sin embargo, ambas soluciones requieren de capas de virtualización y han demostrado tener problemas de hacer aplicaciones que ocupan mucho más espacio [11], además que no resuelven el problema

Sin embargo, a pesar de sus capacidades en términos de reproducibilidad, Nix no ha alcanzado niveles de adopción comparables a los de otras herramientas que abordan problemáticas similares,

¹Sistemas que automatizan la ejecución de tareas repetitivas, usualmente para crear artefactos de software que pueden ser desplegados [2]

como Docker, Conda o VirtualEnv [6]. Esta diferencia no parece atribuirse únicamente a factores técnicos, sino también a aspectos relacionados con su accesibilidad y facilidad de uso. Publicaciones dentro de la comunidad señalan barreras como una documentación compleja, mensajes de error poco interpretables y la necesidad de aprender un lenguaje de programación específico, lo que introduce una curva de aprendizaje considerable. Estudios de usabilidad han reportado ser confusos para usuarios no familiarizados con el paradigma de programación funcional [12]. Estos elementos se enmarcan dentro del ámbito de la experiencia del desarrollador (DX), la cual estudia cómo las herramientas impactan la productividad y la interacción de quienes las utilizan [7]. Y sobre los que ya se han inventando algunas herramientas para evaluarla [13], [14] aunque sigue siendo un campo de estudios emergente.

En este contexto, el presente trabajo busca enfocarse en lenguaje de Nix que el autor de Nix a mencionado contar con problemas de UX [15]. Esto se hará al examinar la experiencia de desarrolladores al utilizar Nix, con el objetivo de identificar problemas de usabilidad y comprender sus implicaciones en la adopción. A partir de este análisis, se propone el desarrollo de una librería que facilite la interacción con Nix mediante un lenguaje de propósito general, buscando mejorar la accesibilidad sin comprometer sus propiedades fundamentales. Se utilizará la técnica de *Programacion Neutral* propuesta [16] para poder hacer la API lo más cercana posible al lenguaje usual con el que los desarrolladores hablarían sobre empaquetar software.

3. Objetivos

3.a. General

Mejorar la experiencia de desarrollo en el manejador de paquetes Nix (DX) al estudiar los problemas de usabilidad de su lenguaje, para diseñar una librería que permita interactuar con nix utilizando Typescript como lenguaje de proposito general.

3.b. Específicos

1. Dirigir estudios de usabilidad en desarrolladores sin utilizando las técnicas cualitativas *pensar-en-alto* para comprender los puntos de dolor que el actual lenguaje de nix presenta.
2. Desarrollar un librería en Typescript y JSii para generar archivos de configuración que sigan el formato flake, cumpliendo las funcionalidades básicas de la librería estandar de Nix para empaquetar software y crear consolas de desarrollo.
3. Comparar la usabilidad y utilidad del lenguaje de Nix contrá la librería desarrollada, en estudiantes de Ciencias de Computación que no tengan experiencia con Nix, con pruebas *pensar-en-alto* y pruebas SDFS-2, IMI y DEXI.

4. Marco Teórico

«Sección vacía»

Referencias

- [1] E. Dolstra, «The Purely Functional Deployment Model», phdthesis, 2006. [En línea]. Disponible en: <https://edolstra.github.io/pubs/phd-thesis.pdf>
- [2] A. Mokhov, N. Mitchell, y S. Peyton Jones, «Build Systems à La Carte», *Executable models for the paper "Build Systems à la Carte"*, vol. 2, n.º ICFP, pp. 79:1-79:29, jul. 2018, doi: 10.1145/3236774.
- [3] J. Malka, S. Zacchiroli, y T. Zimmermann, «Does Functional Package Management Enable Reproducible Builds at Scale? Yes», *Proceedings - 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, MSR 2025*, pp. 775-787, may 2025, doi: 10.1109/MSR66628.2025.00115.
- [4] D. Marakasov, *Repository Statistics*. Accedido: 16 de abril de 2026. [En línea]. Disponible en: <https://repology.org/repositories/statistics/total>
- [5] E. Dolstra y A. Löf, «NixOS: A Purely Functional Linux Distribution», *Proceedings of the ACM SIGPLAN International Conference on Functional Programming, ICFP*, pp. 367-378, sep. 2008, doi: 10.1145/1411204.1411255/SUPPL_FILE/P367-SLIDES.ZIP.
- [6] S. Overflow, *Most Popular Technologies*. Accedido: 16 de abril de 2026. [En línea]. Disponible en: <https://survey.stackoverflow.co/2024/technology>
- [7] F. Fagerholm y J. Münch, «Developer Experience: Concept and Definition», *2012 International Conference on Software and System Process, ICSSP 2012 - Proceedings*, pp. 73-77, 2012, doi: 10.1109/ICSSP.2012.6225984.
- [8] H. Aqasizade, |. E. Ataie, y M. Bastam, «Experimental Assessment of Containers Running on Top of Virtual Machines», ene. 2024, [En línea]. Disponible en: <http://arxiv.org/abs/2401.07539>
- [9] A. Lingayat, R. R. Badre, y A. K. Gupta, «Performance Evaluation for Deploying Docker Containers on Baremetal and Virtual Machine», *Proceedings of the 3rd International Conference on Communication and Electronics Systems, ICCES 2018*, pp. 1019-1023, oct. 2018, doi: 10.1109/CESYS.2018.8723998.
- [10] S. Giallorenzo, J. Mauro, M. G. Poulsen, y F. Siroky, «Virtualization Costs: Benchmarking Containers and Virtual Machines against Bare-Metal», *SN Computer Science*, vol. 2, n.º 5, p. 404–, sep. 2021, doi: 10.1007/S42979-021-00781-8/METRICS.
- [11] M. Zwinger, «Nix: A Solution with Problems», abr. 2026, [En línea]. Disponible en: <https://arxiv.org/abs/2604.11398v1>
- [12] fricklerhandwerk, «Usability Studies - Development / Documentation - NixOS Discourse». [En línea]. Disponible en: <https://discourse.nixos.org/t/usability-studies/21404>
- [13] K. Kuusinen, «Are Software Developers Just Users of Development Tools? Assessing Developer Experience of a Graphical User Interface Designer», *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, pp. 215-233, ago. 2016, doi: 10.1007/978-3-319-44902-9_14.
- [14] T. de Moura Parracho, R. O. Zacarias, M. C. da Rocha Seruffo, y R. P. dos Santos, «I Didn't Find What I Wanted - How Do Developers Consume Information in Software Ecosystems Portals?»,

ACM International Conference Proceeding Series, vol. 1, pp. 143-150, may 2023, doi:
10.1145/3592813.3592899.

- [15] E. Dolstra, «The Future of Nix», 2018.
- [16] J. F. Pane y B. A. Myers, «(PDF) More Natural Programming Languages and Environments», *ResearchGate*. 2006. doi: 10.1007/1-4020-5386-X_3.